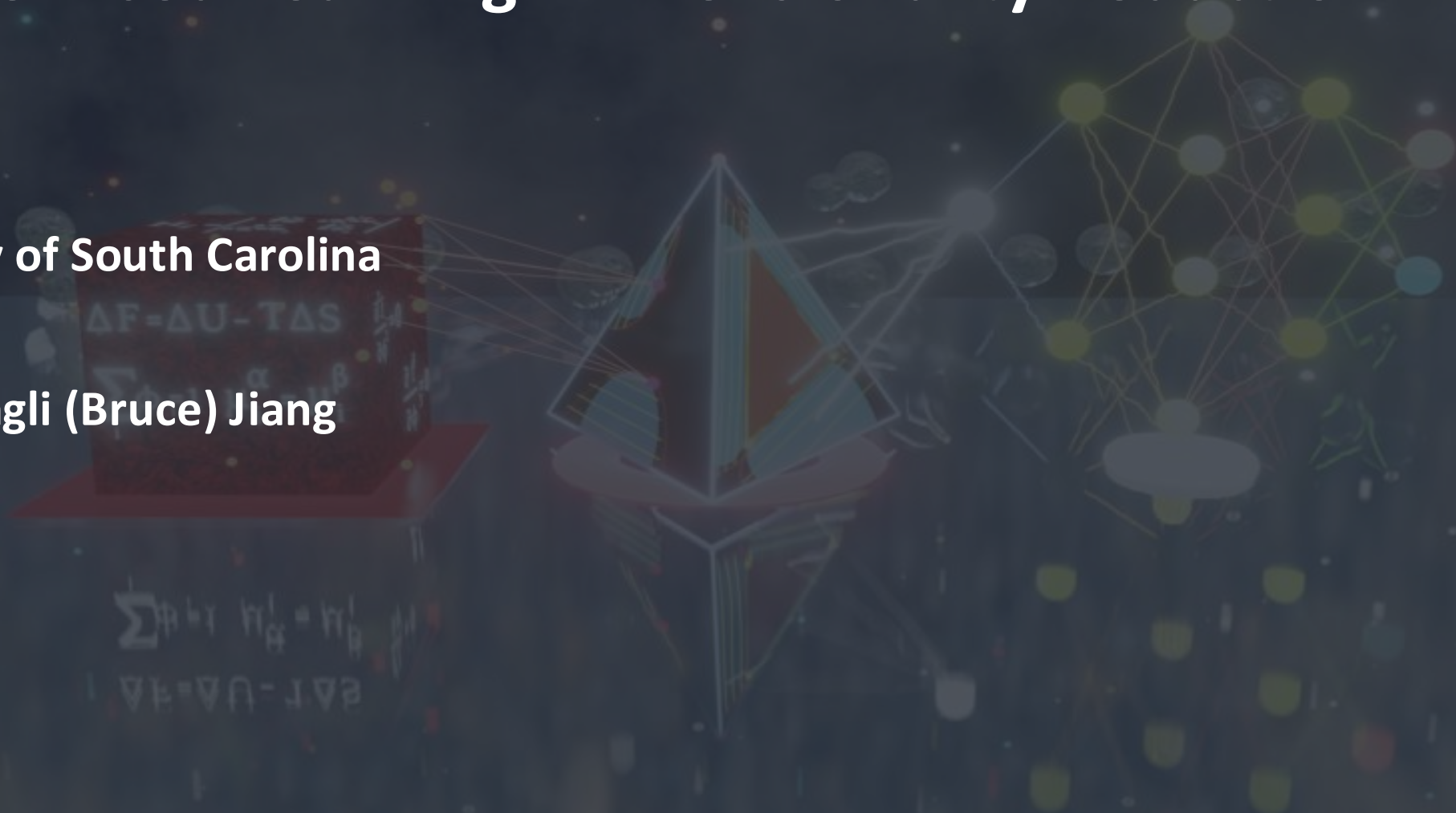


Unsupervised Learning: Dimensionality Reduction

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



From Lecture 19

Neural Networks III: NN in Practice

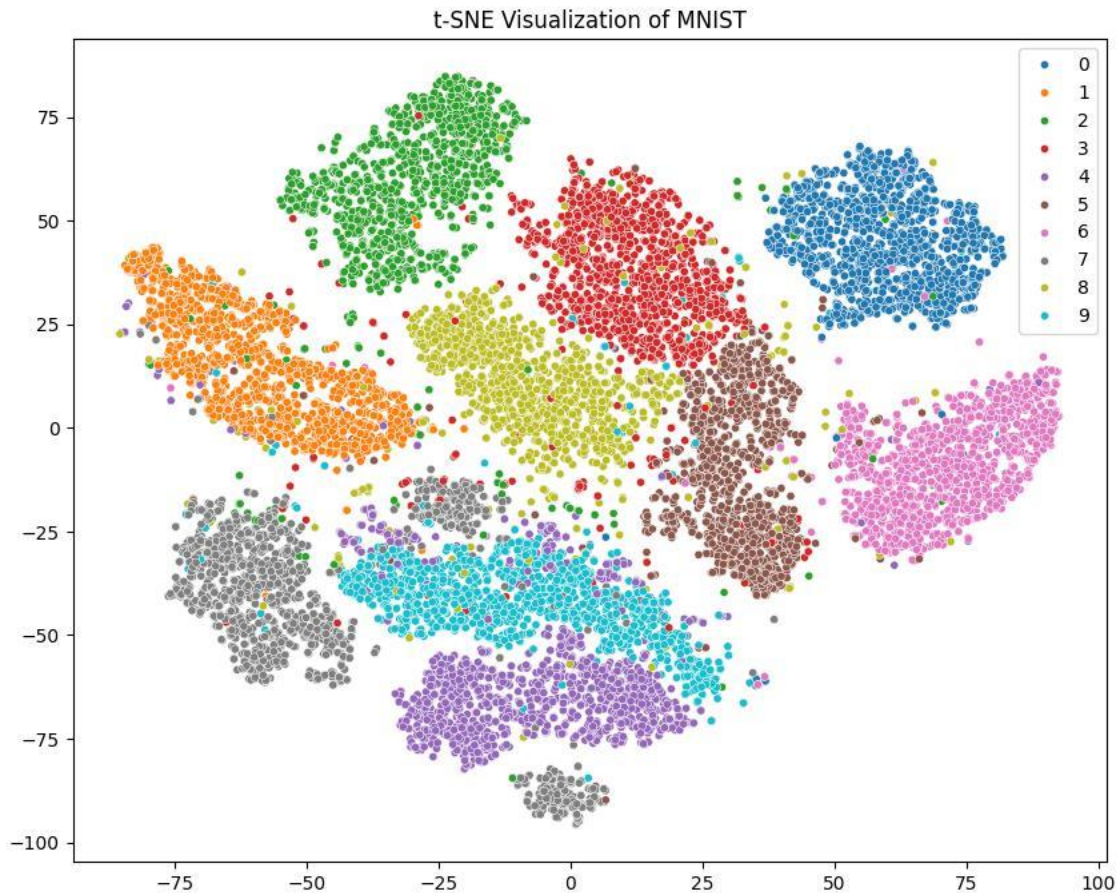
- *Epochs, batches, and learning rate*
- *Optimizers: SGD and Adam*
- *Training curves, overfitting, and regularization*

In this lecture, you will learn

- *Finding hidden structure in data without target labels*
- *Using PCA and explained variance for dimensionality reduction*
- *Extending PCA to nonlinear structure with kernel methods*

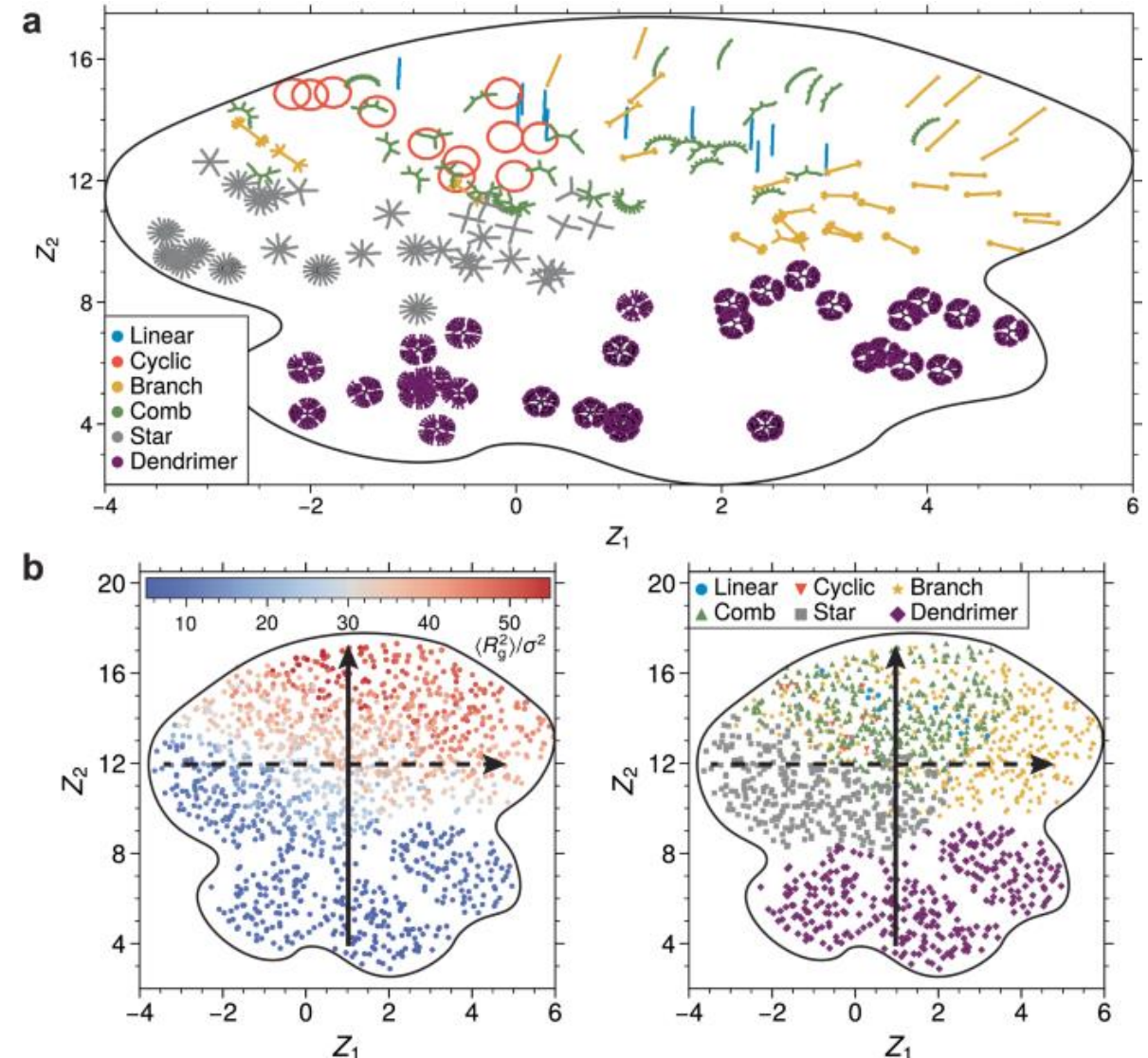
Unsupervised Learning: Can We Find Structure Without Answers?

70,000 handwritten digits



- Imagine giving the computer thousands of handwritten images **without telling it which are 0, 1, 2, ...**
- *Can it discover that some images are similar?*

Underlying patterns in polymer architectures



Why should we use dimensionality reduction?



*Do we need all of these features to accurately predict the output?
Can we get away with only using a subset of the feature space?*

Considerations

- Model trains faster since it has fewer dimensions.
- Make the model simpler for researchers to interpret/visualize.
- Improve model accuracy due to less misleading or noisy data.

Types of Dimensionality Reduction Techniques

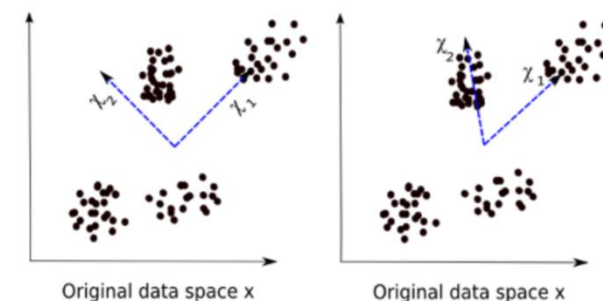
Most DR techniques fall into a few categories

- **Feature Elimination and Extraction**

*we have been working with this already a bit
in general, the goal is to systematically remove
uninformative/redundant variables*

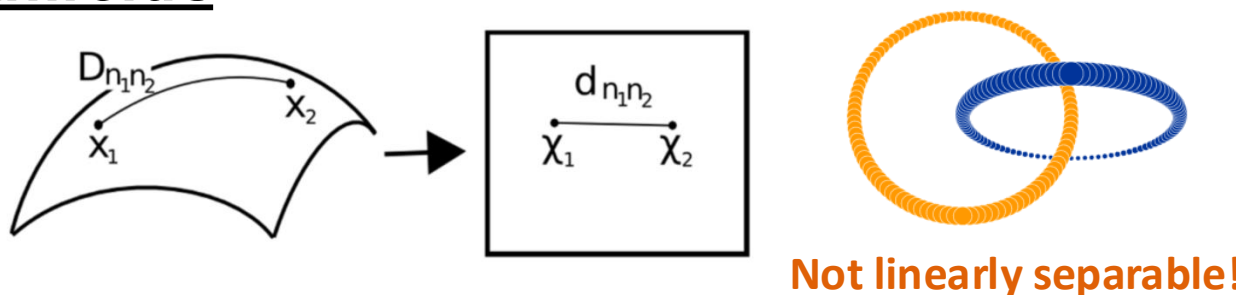
- **Linear Transformations/Components**

*here the strategy is to find a linear
transformation of your feature space that
results in more “useful” set of coordinates*



- **Non-linear Transformations & manifolds**

*in this case, we usually perform some non-
linear transformation and project the features
onto a lower-dimensional manifold*



Feature Elimination and Extraction

From a feature set, we are looking for a subset of features to use:

- Remove features with too many missing values (**Missing Value Ratio**)
- Remove features that exhibit small variance (**Low-variance Filter**)
- Remove highly correlated features (e.g., using Pearson's r) (**High Correlation filter**)
- Assess **feature importance** to model predictions
 - by using e.g., **Random Forest, SHAP**
 - by systematically removing features (**Backward Feature Elimination**)
 - by systematically adding features (**Forward Feature Selection**)

These are all very simple, powerful strategies that may be complementary to other methods → always worth considering!

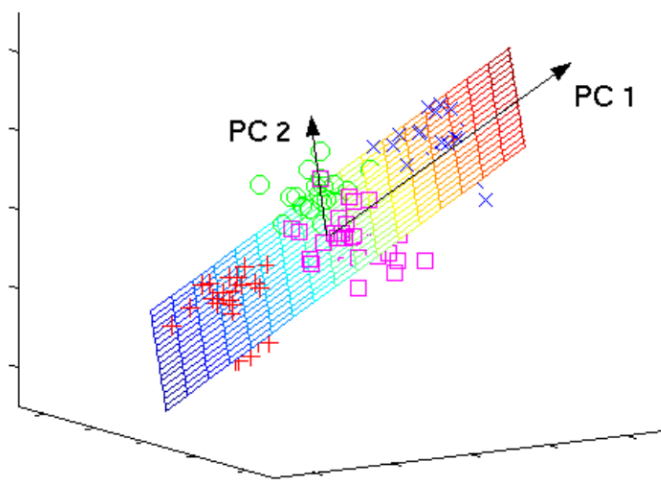
Principal Component Analysis (PCA)

PCA is the “canonical” and most common DR technique

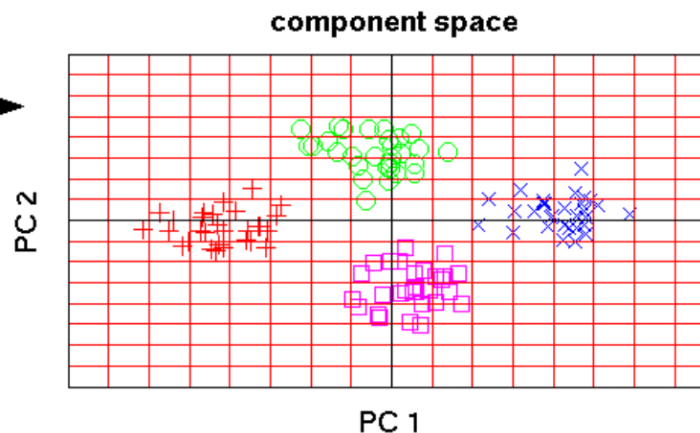
Important features of PCA

- *PCA assumes linear relationships between variables.*
- *PCA is scale dependent (features with larger values look more important)*
- *PCA looks at variance in the feature data (it can be important to preprocess the data via normalization, mean-centering, or scaling).*
- *PCA is one of the central applications of SVD*
- *PCA can be used for preprocessing for other DR techniques*

original data space



PCA



The basic goal:

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n; \mathbf{x}_i \in \mathbb{R}^d$$

$$\downarrow W$$

$$\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n; \mathbf{z}_i \in \mathbb{R}^k$$

$$k \ll d$$

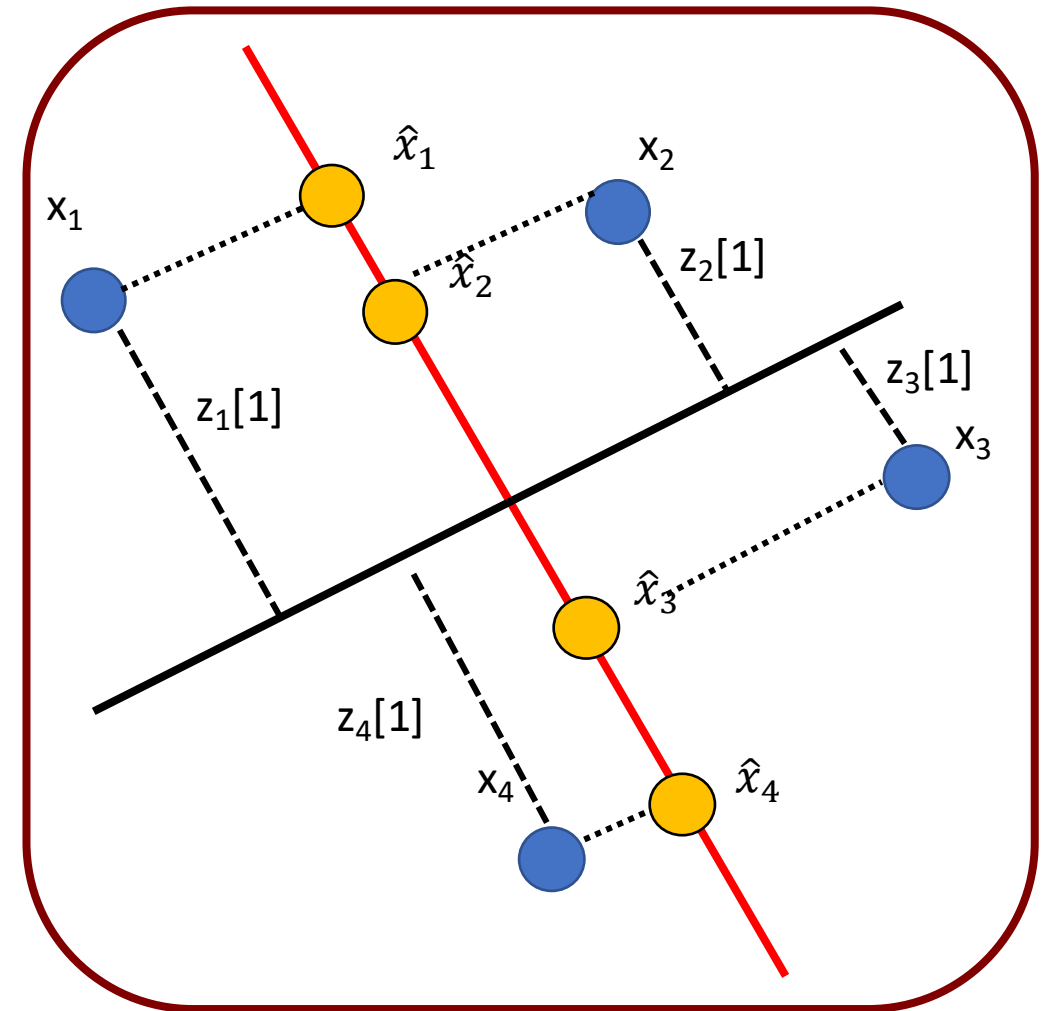
Principal Component Analysis (PCA)

The big idea:

1. We want to identify an **orthonormal** basis to represent $\mathbf{x}_i - \boldsymbol{\mu} \rightarrow d$ -dimensional basis
2. We will only pick k of the d basis vectors
3. All of the points will then be represented in a k -dimensional subspace spanned by $\mathbf{w}_1, \dots, \mathbf{w}_k$

$$\mathbf{x}_i - \boldsymbol{\mu} = \sum_{j=1}^d \mathbf{z}_i[j] \cdot \mathbf{w}_j \quad \text{exact reconstruction}$$

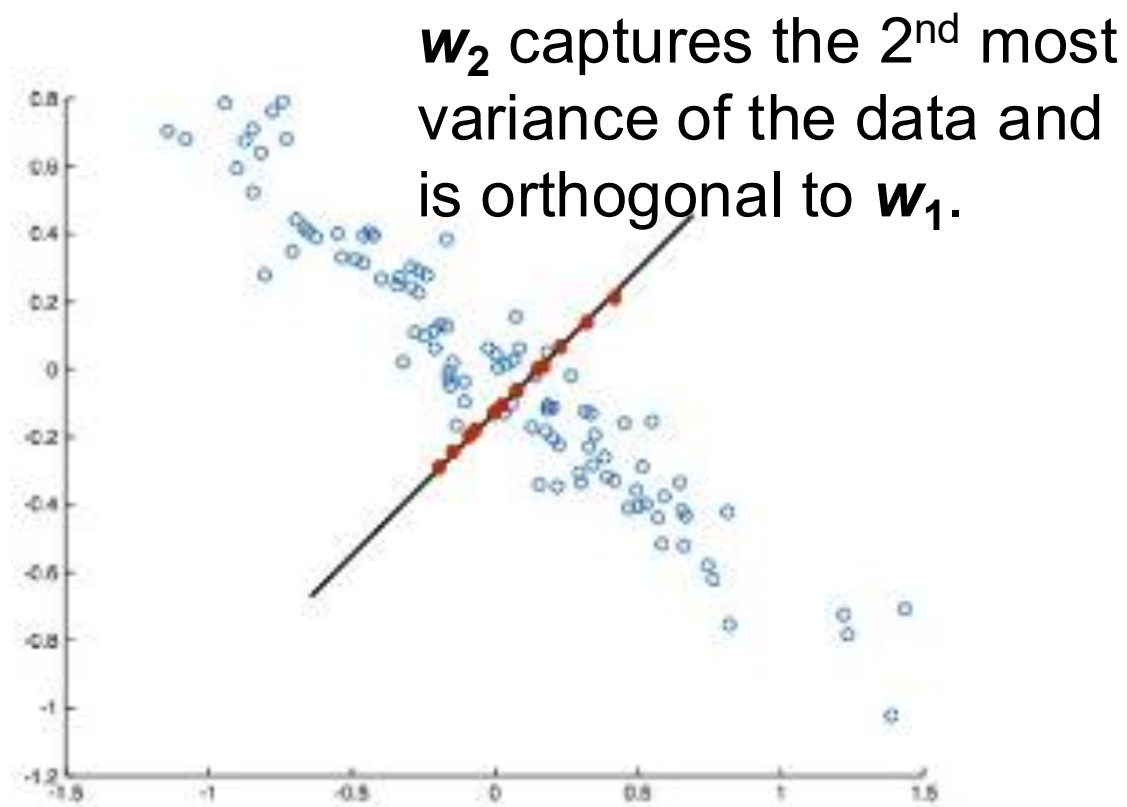
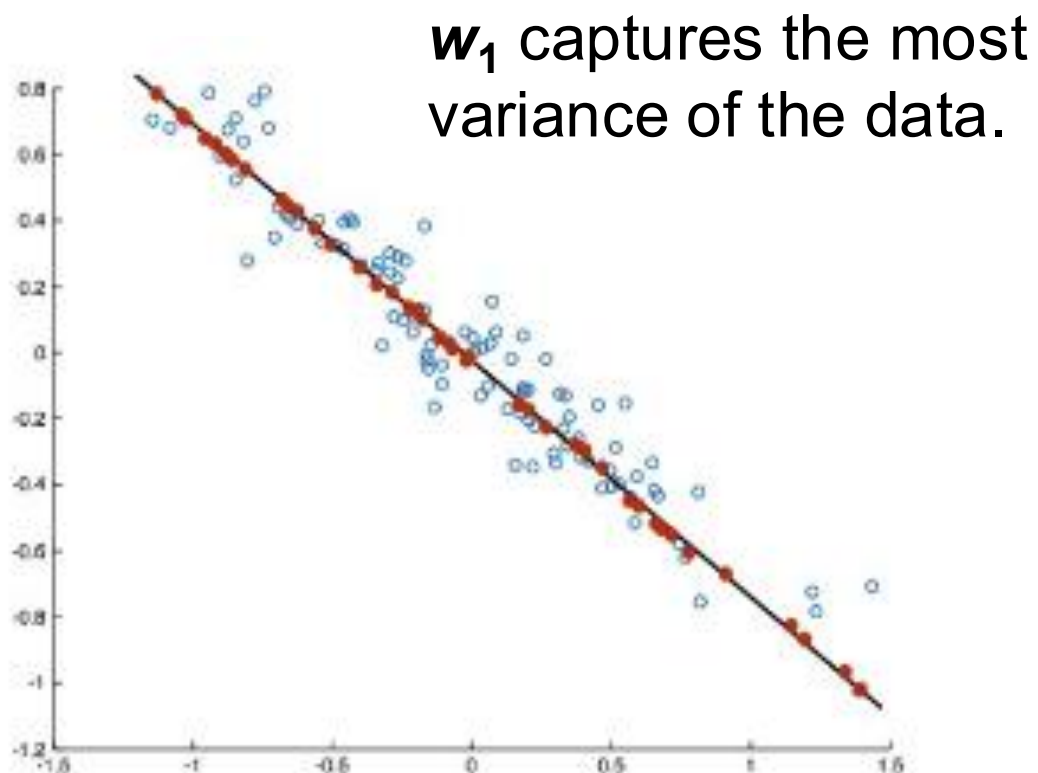
$$\hat{\mathbf{x}}_i - \boldsymbol{\mu} = \sum_{j=1}^k \mathbf{z}_i[j] \cdot \mathbf{w}_j \quad \text{approximate reconstruction}$$



Algorithm $\rightarrow$ how to choose $\mathbf{W}$ and which to keep

PCA: How to pick basis vectors

Idea: Find the directions along which the data is maximally spread (highest variance).



PCA: How to pick basis vectors

Idea: Find the directions along which the data is maximally spread (highest variance).

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^T \in \mathbb{R}^{n \times d} \quad \boldsymbol{\mu} = \left[\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i[1], \dots, \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i[d] \right]^T$$

$$\mathbf{B} = \mathbf{X} - \mathbf{1}_n \boldsymbol{\mu}^T$$

- mean-subtracted data

$$\mathbf{C} = \frac{1}{n-1} \mathbf{B}^T \mathbf{B}$$

- covariance matrix

$$\mathbf{w}_1 = \arg \max_{\|\mathbf{w}\|_2=1} \mathbf{w}^T \mathbf{C} \mathbf{w}$$

- first principal component

subsequent components can be determined by orthogonalization and repetition

PCA: How to pick basis vectors

Idea: Find the directions along which the data is maximally spread (highest variance).

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^T \in \mathbb{R}^{n \times d} \quad \boldsymbol{\mu} = \left[\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i[1], \dots, \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i[d] \right]^T$$

$$\mathbf{B} = \mathbf{X} - \mathbf{1}_n \boldsymbol{\mu}^T$$

- mean-subtracted data

$$\mathbf{C} = \frac{1}{n-1} \mathbf{B}^T \mathbf{B}$$

- covariance matrix

$$\mathbf{B} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{W}^T$$

In practice, we obtain all principal components at once using eigendecomposition or SVD.

PCA: additional details/realizations

- Find the unit direction $\mathbf{w}$ that maximizes the variance. This is a constrained optimization problem, so we use a Lagrange multiplier.

$$\mathbf{w}_1 = \arg \max_{\|\mathbf{w}\|_2=1} \mathbf{w}^T \mathbf{C} \mathbf{w} \quad L(\mathbf{w}, \lambda) = \mathbf{w}^T \mathbf{C} \mathbf{w} - \lambda (\mathbf{w}^T \mathbf{w} - 1)$$

- Set the derivative of the Lagrangian to zero:

$$\frac{\partial L}{\partial \mathbf{w}} = 2\mathbf{C}\mathbf{w} - 2\lambda\mathbf{w} = \mathbf{0}$$

- This gives an eigenvalue equation. For a unit eigenvector $\mathbf{w}$, the variance along that direction equals its eigenvalue λ .

$$\mathbf{C}\mathbf{w} = \lambda\mathbf{w} \quad \mathbf{w}^T \mathbf{C} \mathbf{w} = \lambda \mathbf{w}^T \mathbf{w} \quad \mathbf{w}^T \mathbf{C} \mathbf{w} = \lambda$$

- Therefore, PCA ranks the eigenvectors of $\mathbf{C}$ by their eigenvalues. Largest eigenvalue gives PC1, second largest gives PC2, and so on.

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \quad \mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_d$$

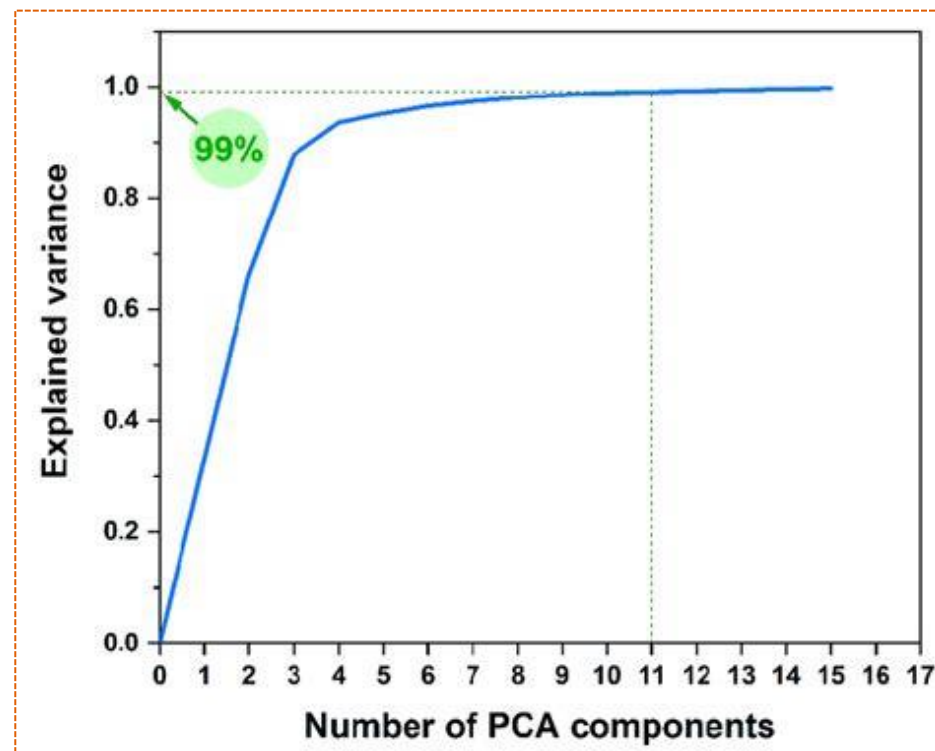
PCA: How Much Variance Is Explained?

It is natural to ask how much of the information in a data set is lost by projecting the observations onto a small subset of principal components.

The amount of **variance explained** can be analytically related to the eigenvalues of the covariance matrix via:

$$\text{EVR}(j) = \frac{\lambda_j}{\sum_{j=1}^d \lambda_j}$$

$$\text{Cumulative EVR}(k) = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^d \lambda_j}$$



PCA: Application to Visualizing Chemical Space

Open Macromolecular Genome: Generative Design of Synthetically Accessible Polymers

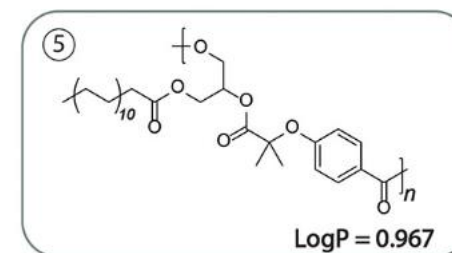
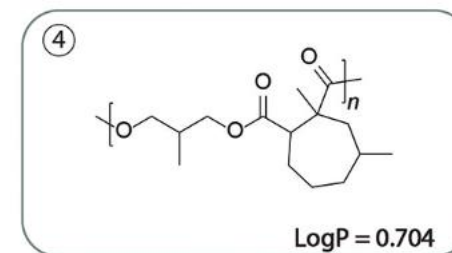
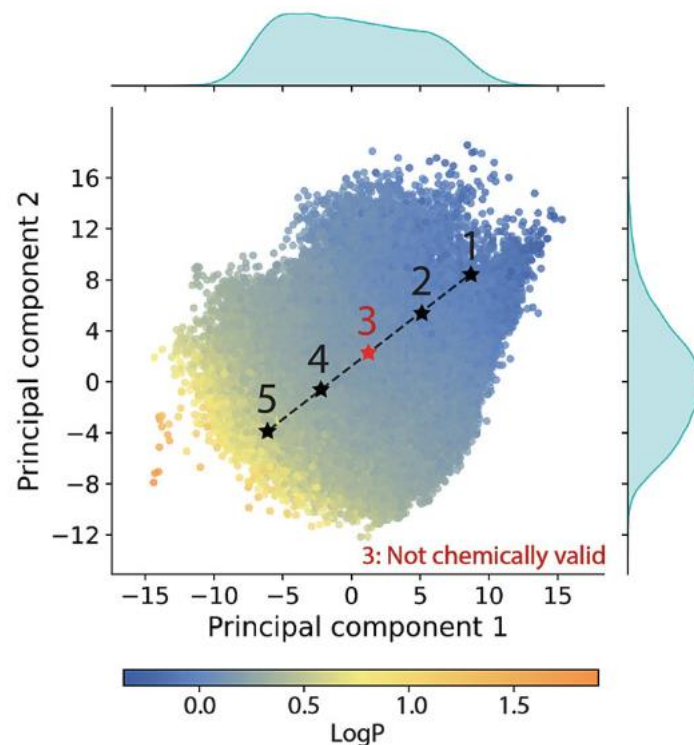
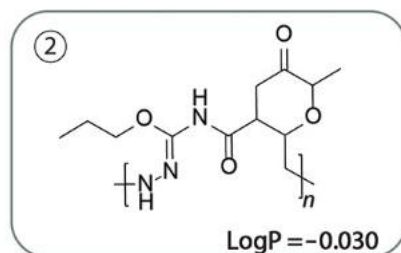
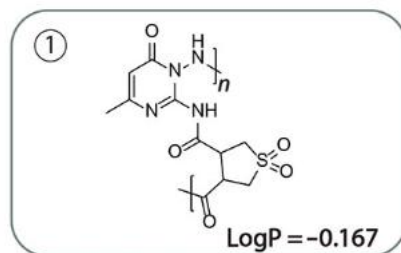
Seonghwan Kim, Charles M. Schroeder, and Nicholas E. Jackson*



Cite This: *ACS Polym. Au* 2023, 3, 318–330

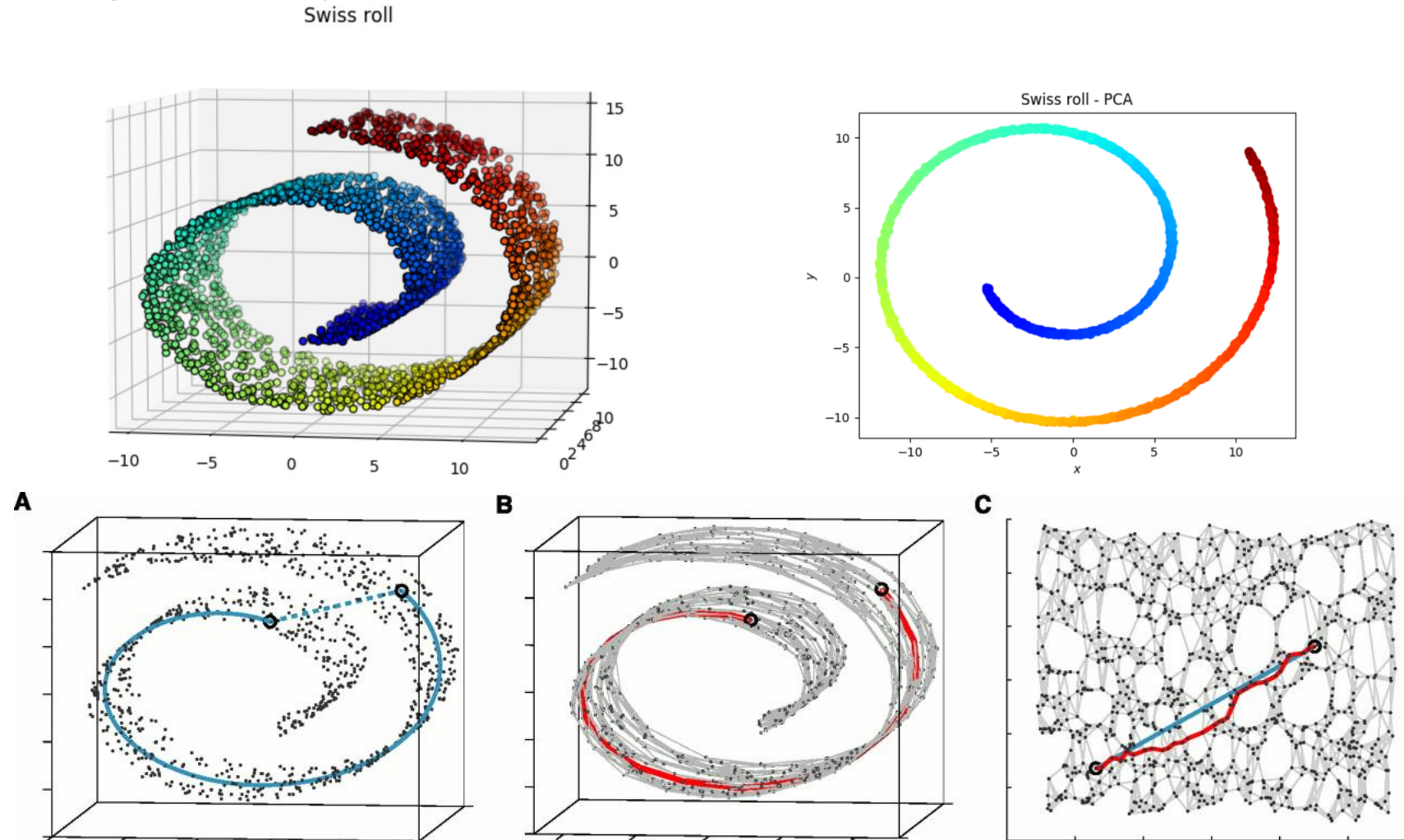


Read Online



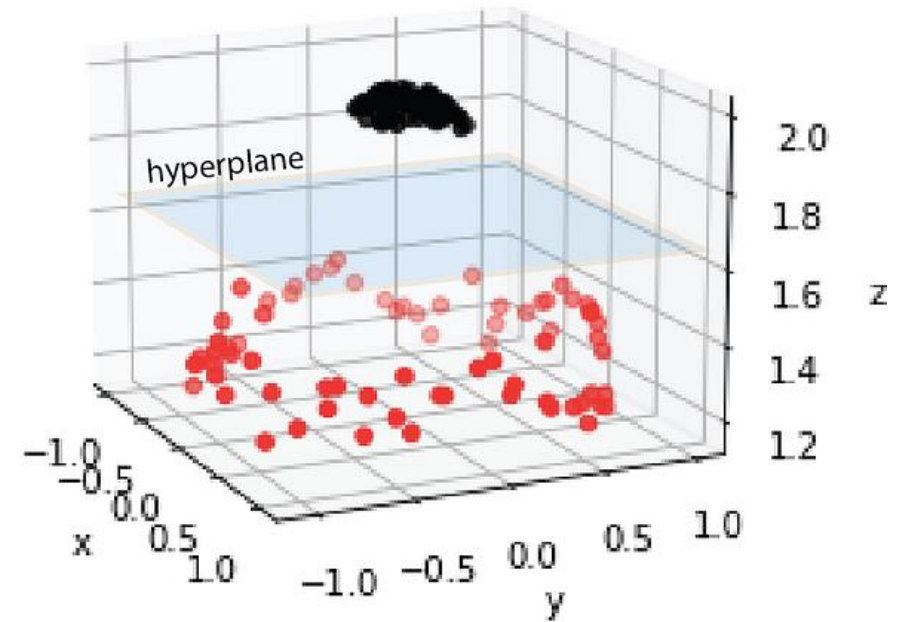
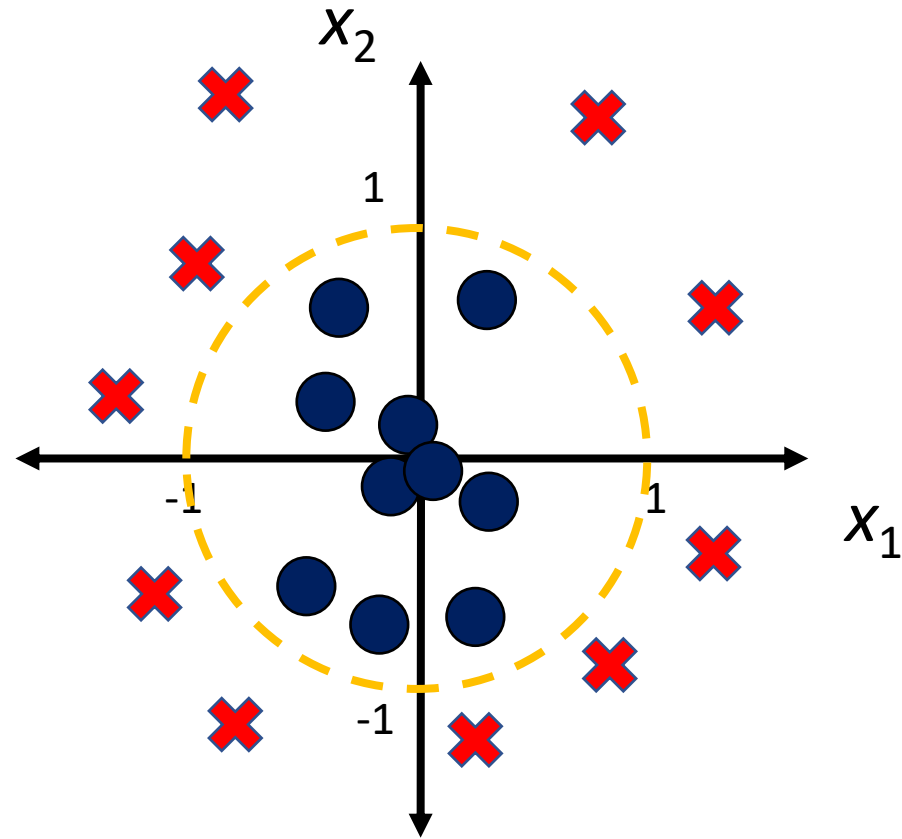
When PCA Fails: Nonlinear Manifolds

PCA generally fails to detect low-dimensional manifolds



Tenenbaum, J. B.; de Silva, V. & Langford, J. C. A Global Geometric Framework for Nonlinear Dimensionality Reduction *Science*, **2000**, 290, 2319-2323

A (relatively) Simple Modification: Kernel PCA

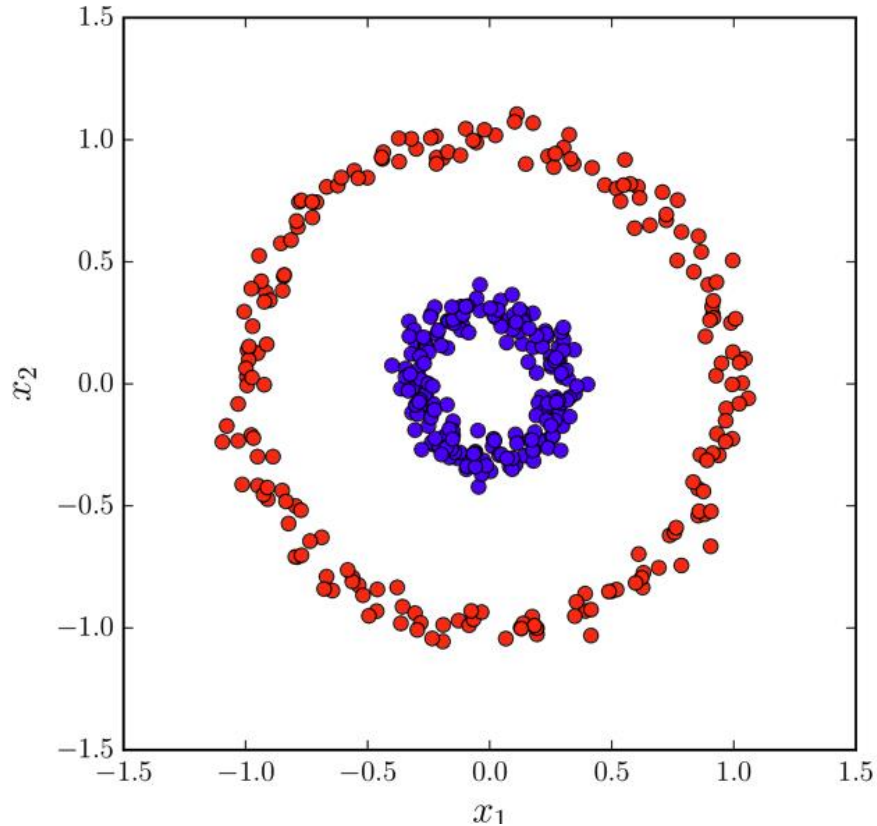


$$f(x_1, x_2) = \frac{1}{1 + e^{-z}}, \quad z = x_1^2 + x_2^2$$

Nonlinear boundary in the original space $\rightarrow$ linear boundary in the transformed space.

A (relatively) Simple Modification: Kernel PCA

Consider the following...



**What happens if we
do PCA here?**

The Kernel Trick

- Implicitly map the data into a nonlinear feature space
- Instead of computing the transformed coordinates directly, calculate similarities between pairs of samples
- The kernel gives the inner products in the transformed feature space
- Perform PCA on the resulting kernel matrix to capture nonlinear structure

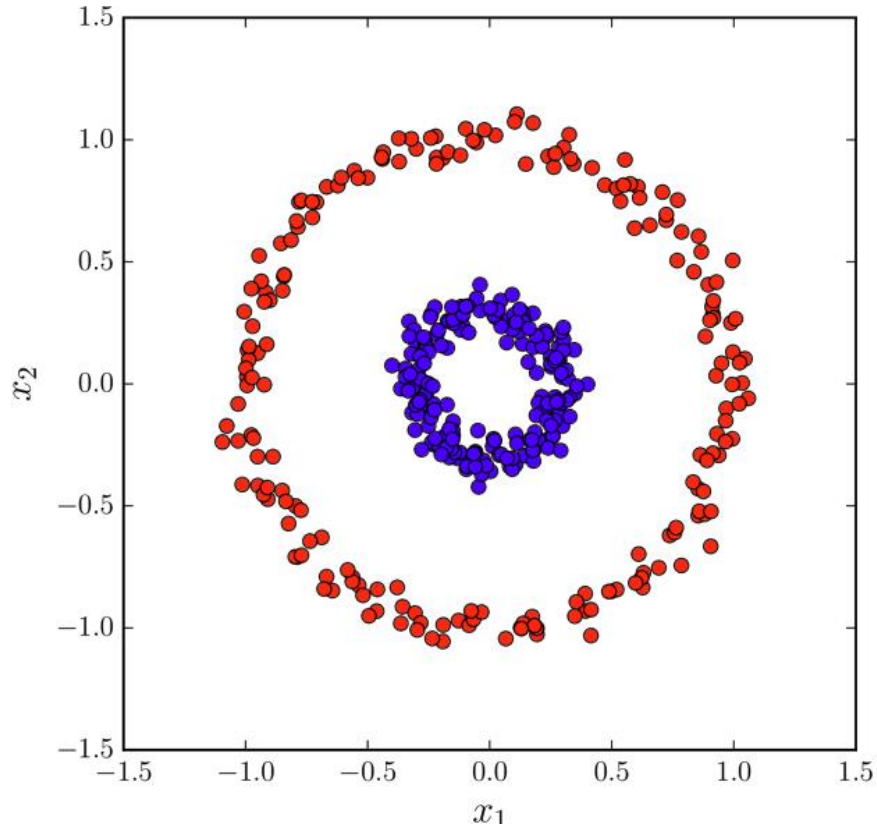
RBF kernel

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

Note: In kernel PCA, the kernel matrix must be centered so the transformed data have zero mean in feature space.

A (relatively) Simple Modification: Kernel PCA

Consider the following...



**What happens if we
do PCA here?**

Algorithm to implement Kernel PCA:

1. Compute the Kernel matrix, $\mathbf{K}$.

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j); \quad \mathbf{K} \in \mathbb{R}^{n \times n}$$

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp \left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2 \right)$$

2. Center the kernel matrix (like normal feature scaling, but in kernel space) using

$$\mathbf{K}_c = \mathbf{H} \mathbf{K} \mathbf{H} \quad \mathbf{H} = \mathbf{I} - \frac{1}{n} \mathbf{1} \mathbf{1}^T$$

3. Solve the eigenvalue problem

$$\mathbf{K}_c \mathbf{v}_j = \lambda_j \mathbf{v}_j$$

Interactive Python Time

20_DR_PCA.ipynb

In this lecture, we learned

- *Finding hidden structure in data without target labels*
- *Using PCA and explained variance for dimensionality reduction*
- *Extending PCA to nonlinear structure with kernel methods*

Next lecture:

- *Visualizing nonlinear structure with t-SNE and UMAP*
- *Measuring similarity and distance between data points*
- *Grouping unlabeled data using clustering methods*